Отчет о выполнении лабораторной работы по pthread

Антонов Артем Максимович 23КНТ3

October 21, 2026

1 Множество Мондельброта

1.1 Математический алгоритм

Множество Мандельброта — это совокупность всех чисел $c \in \mathbb{C}$, для которых последовательность

$$z_0 = 0, \qquad z_{n+1} = z_n^2 + c \tag{1}$$

является ограниченной для всех значений $n \in \mathbb{N}$. Иными словами, число c принадлежит множеству Мандельброта тогда и только тогда, когда

$$(\exists A \in \mathbb{R})\ (\forall n \in \mathbb{N})\ \ |z_n| < A. \tag{2}$$

Представим z^2 как комплексное число, возведенное в квадрат:

$$z = x + yi \tag{3}$$
$$z^2 = (x+yi)^2 = x^2 + 2xyi + y^2 i^2 = x^2 + 2xyi - y^2 = (x^2 - y^2) + 2xyi \tag{4}$$

Прибавим к получившемуся выражению $c = a + bi$

$$z^2 = (x^2 - y^2) + 2xyi + a + bi = (x^2 - y^2 + a) + (2xyb)i \tag{5}$$

В итоге получаем

$$z_{n+1} = (x^2 - y^2 + a) + (2xyb)i \tag{6}$$

Отсюда,

$$x_{n+1} = x_n^2 - y_n^2 + a \tag{7}$$
$$y_{n+1} = 2x_n y_n b \tag{8}$$

Теперь рассмотрим условие принадлежности числа c к множеству Мондельброта:

$$|z_n| \leq 2 \tag{9}$$

Так как модуль комплексного числа это расстояние от точки до начала координат, можно z представить так:

$$z_n = \sqrt{x_n^2 + y_n^2} \tag{10}$$

Далее подставим это в исходное выражение и возведемв квадрат

$$\sqrt{x_n^2 + y_n^2} < 2 \tag{11}$$
$$x_n^2 + y_n^2 < 4 \tag{12}$$

Итого, получаем, что при $x_n^2 + y_n^2 < 4$ точка c принадлежит множеству Мондельброта.

Соответственно, при $x_n^2 + y_n^2 > 4$ - точка c гарантированно не принадлежит множеству Мондельброта.

В итоге всех преобразований мы получили 3 основные формулы, которые будут использоваться в программе.

Для нахождения новый x и y на каждой итерации цикла проверки числа $c = a + bi$:

$$x_{n+1} = x_n^2 - y_n^2 + a \tag{13}$$
$$y_{n+1} = 2x_n y_n b \tag{14}$$

И условие выхода из цикла проверки числа c:

$$x_n^2 + y_n^2 > 4 \tag{15}$$

1.2 Обзор кода

Программа реализует следующий алгоритм нахождения точки множества Мондельброта:

1. С помощью генератора псевдослучайных чисел генерируем a и b и подгоняем их в диапазон точек.

2. Итерируемся N раз, на каждой итерации генерируя x и y по формулам, которые были выведены в прошлой главе

3. Проверяем условия выхода из цикла, полученное в прошлой главе, если оно выполняется - ставим флаг и break

4. Проверяем флаг, если мы видим значение, означающее, что мы не вышли из цикла с помощью break (0), то записываем точку в глобальных массив точек.

Этот алгоритм выполняется каждым потоком. Поток находит свое определенное количество точек, которое расчитывается по следующей формуле:

```
int n_points = npoints / nthreads + (my_rank < npoints % nthreads ? 1 : 0);
```

1.3 Замеры производительности

Замеры выполнялись на ubuntu 22.04 под WSL2. Процессор Intel Core i5-10400F - 6 ядер и 12 потоков.

Количество итераций N в каждом запуске - 1000.

Table 1: Результаты замеров

Run number	NPOINTS	Threads count	AVG	MIN
15	1 000 000	1	6.3862	6.2632
15	1 000 000	2	3.2562	3.1483
15	1 000 000	4	1.7345	1.6990
15	1 000 000	8	0.9184	0.9019
15	1 000 000	12	0.7091	0.6618
15	1 000 000	16	0.7943	0.7655
15	1 000 000	24	0.6934	0.6736

2 Своя реализация `rwlock`

2.1 Обзор кода

В рамках лабораторной работы была реализована структура `my_rwlock_t` и основные ее функции `rwlock`:

- `int my_rwlock_init(my_rwlock_t* loc)`
- `int my_rwlock_destroy(my_rwlock_t* loc)`
- `int my_rwlock_rdlock(my_rwlock_t* loc)`
- `int my_rwlock_rwlock(my_rwlock_t* loc)`
- `int my_rwlock_unlock(my_rwlock_t* loc)`

2.1.1 `my_rwlock_init`

Функция `my_rwlock_init` инициализирует поля структуры с помощью соответствующих стандартных функций, при ошибке инициализации возвращает код ошибки.

2.1.2 `my_rwlock_destroy`

Вызывает соответствующие `destroy` функции для объектов структур `pthread_mutex_t` и `pthread_cond_t`

2.1.3 `my_rwlock_rdlock`

Функция реализует механизм блокировки читателей. Для любых операций с внутренними переменными используются мьютекс. Соответственно, в начале функции мы блокируем мьютекс. Далее мы увеличиваем каунтер ожидающих читателей и запускаем цикл while с проверкой того, есть ли активный писать или есть ли ожидающие писатели, если условие выполняется - усыпляем поток, иначе уменьшаем количество ожидающих читателей и увеличиваем количество читателей и разблокируем мьютекс.

2.1.4 `my_rwlock_wrlock`

Функция реализует механизм блокировки писателей. Общая идея та же, мы увеличиваем каунтер и заходим в цикл, где условием является наличие активных писателей или наличие читателей. Если условие выполняется - засыпаем, иначе ставим флаг активного писателя и уменьшаем каунтер ожидающих писателей

2.2 Замеры производительности

Table 2: Сравнение STDLIB и LAB при size = 1000

Workload	Threads	STDLIB		LAB	
		min	average	min	average
read99.9	1	0.0516	0.0561	0.0547	0.0582
	2	0.0311	0.0324	0.0376	0.0392
	4	0.0245	0.0254	0.0392	0.0401
	8	0.0254	0.0259	0.0477	0.0486
	16	0.0274	0.0316	0.0590	0.0613
	24	0.0247	0.0265	0.0701	0.0725
mixed80	1	1.0249	1.1153	1.0226	1.1096
	2	−0.8236	1.2810	1.2634	1.3065
	4	1.6442	1.6931	−0.6951	1.4461
	8	1.8803	1.9129	−0.0671	2.1165
	16	2.4456	2.4790	0.9084	3.0165
	24	0.6888	2.8203	3.9499	4.0046
write50	1	0.0659	3.2365	1.3563	3.4658
	2	2.9325	4.9232	1.8000	4.0062
	4	3.2489	5.1671	2.2969	4.7139
	8	3.2868	5.4028	3.1590	4.9868
	16	5.3473	7.2712	5.2359	7.1234
	24	6.7542	8.5350	5.7154	8.7316